

BAYESIAN OPTIMIZATION CENTERED DOMAIN RANDOMIZATION VIA ENTROPY MAXIMIZATION

Suhaima Ahmad, Fata Asghar Zadniazi, Kevaan Buch

Department of Mathematical & Computational Sciences

University of Toronto

Mississauga, ON L5L 1C6

{suhaima.ahmad, f.asghar Zadniazi, kevaan.buch}@mail.utoronto.ca

<https://github.com/FataNiazi/BODREM.git>

ABSTRACT

Training Reinforcement Learning (RL) agents in simulation is a popular approach, as real-world data can be prohibitively expensive. However, transferring such agents to the real world is hindered by the reality gap, the mismatch between simulated and real dynamics. Domain Randomization (DR) Tobin et al. (2017) addresses this issue by training the agent over a randomized environment. However, designing an effective distribution for the randomization parameters remains a challenge.

The current approaches either form the training distribution without target-domain feedback (e.g., DORAEMON Tiboni et al. (2024)) or use Bayesian Optimization (BO) to align the distribution with the target domain, but do not explicitly promote robustness (e.g., BayRn Muratore et al. (2021)). We propose BO-Centred DORAEMON, an approach that separates the center and spread of the training distribution. The center of the domain distribution is optimized using BO over sparse rollouts in the target simulator, while the spread is expanded via entropy maximization in a cheaper surrogate simulator. This separation allows the training distribution to better align with the target environment while maintaining robustness to dynamics variations.

1 INTRODUCTION

Reinforcement Learning (RL) has shown strong performance in simulated environments, where agents can be trained efficiently and safely. However, deploying RL policies in real-world environments remains a significant challenge due to the *reality gap*, which refers to the mismatch between simulated dynamics and real-world dynamics. Small differences in parameters such as friction, mass, sensor noise, and latency can cause policies trained in simulation to perform poorly when transferred to real-world systems.

Domain Randomization (DR) is a commonly used approach to address this problem. In domain randomization, the agent is trained across a distribution of simulated environments with varying parameters, with the goal of learning a policy that is robust to environmental variation. If the real-world environment lies within the support of this distribution, the trained policy can transfer successfully. However, the main challenge in domain randomization is determining how to choose the distribution of environment parameters. A distribution that is too narrow leads to poor generalization, while a distribution that is too wide can make training unstable and inefficient.

Recent methods attempt to learn this distribution automatically. Some approaches expand the distribution to increase robustness without using real-world feedback, while others use Bayesian Optimization (BO) to align the distribution with the target domain using limited real-world data. However, these approaches typically focus on either robustness or alignment, and do not explicitly separate these two objectives.

In this paper, we propose *Bayesian Optimized Randomization of Domain via Entropy Maximization (BORDERM)*, a method that separates the domain randomization distribution into two components:

a center and a spread. The center of the distribution is optimized using Bayesian Optimization based on sparse feedback from the target environment, while the spread is expanded using entropy maximization in a surrogate simulator, subject to a minimum performance constraint to ensure stable training. By separating the alignment and robustness objectives, the proposed method aims to improve sim-to-real transfer while maintaining stable and efficient training.

2 RELATED WORK

Sim-to-sim is a widely used evaluation method for assessing a deep learning model’s generalizability: a model is trained in one simulation environment and evaluated in another. The ultimate goal of sim-to-sim is to transfer the model to the real world, also known as sim-to-real. Sim-to-sim helps assess a model’s generalization performance before real-world testing, thereby significantly reducing the resources and costs required. Furthermore, evaluating an RL agent in the real world is often infeasible due to cost and safety concerns, making simulation-based training the preferred approach and sim-to-real transfer an unavoidable challenge. We organize prior work into three main categories: domain randomization, entropy-based approaches for robustness and adaptive domain distribution learning using Bayesian optimization.

2.1 DOMAIN RANDOMIZATION

Domain Randomization (DR) is a widely used technique for bridging the sim-to-real gap by training policies over a distribution of environments with varying dynamics parameters. Instead of relying on a single simulator, DR exposes the agent to diverse conditions, enabling it to generalize to unseen environments. Early work by Tobin et al. (2017) demonstrated that randomizing simulation parameters can significantly improve transfer performance.

Despite its success, standard DR relies on manually designed distributions. This introduces a key limitation: the performance of the learned policy is highly sensitive to the choice of distribution. Narrow distributions lead to overfitting, while overly broad distributions make learning unstable and inefficient. This motivates the need for methods that can automatically adapt the domain distribution during training.

2.2 DOMAIN RANDOMIZATION VIA ENTROPY MAXIMIZATIONS

A more recent work asks how to systematically enlarge the training distribution while preserving learnability. Tiboni et al. (2024) propose DORAEMON (D**O**main R**A**ndomization via E**N**tropy M**A**ximization). That is maximize the entropy of the dynamics distribution while enforcing a minimum success probability for the current policy. This produces policies that are adaptive across broader dynamics ranges and can transfer successfully even without real world feedback during distribution shaping. The significance of DORAEMON is that it makes the robustness objective explicit. Earlier adaptive DR methods often widen the distribution heuristically or indirectly, whereas DORAEMON directly optimizes for diversity through entropy. At the same time, the performance constraint prevents the randomization process from collapsing training by expanding too aggressively. This balance between diversity and feasibility makes entropy maximization especially appealing for sim-to-real settings where robustness to unknown dynamics is needed.

2.3 BAYESIAN OPTIMIZATION FOR DOMAIN ADAPTATION

Bayesian Optimization (BO) provides a complementary approach by adapting the training distribution using sparse feedback from the target domain. Methods such as Bayesian Domain Randomization (BayRn) Muratore et al. (2021) treat the selection of domain parameters as an optimization problem, where the objective is to maximize performance in the target environment.

BO is particularly effective in this setting because target evaluations are expensive. By building a probabilistic model of the objective function, BO can efficiently explore the parameter space using a limited number of evaluations. Information-theoretic variants such as Max-value Entropy Search Wang & Jegelka (2017) further improve efficiency by selecting evaluation points that reduce uncertainty about the optimum.

However, BO based methods primarily focus on identifying a high-performing region of the parameter space, effectively optimizing the *center* of the distribution. They do not explicitly promote robustness across a wide range of dynamics, which can lead to policies that are sensitive to perturbations around the optimized parameters.

2.4 RESEARCH GAP

Existing approaches to domain randomization tend to focus on either robustness or alignment, but not both simultaneously. Entropy-based methods such as DORAEMON Tiboni et al. (2024) improve robustness by expanding the distribution but remain target-agnostic. In contrast, BO-based methods such as BayRn Muratore et al. (2021) improve alignment with the target domain but often produce narrowly tuned solutions.

This reveals a key gap: there is no unified framework that jointly optimizes both the *center* and the *spread* of the domain randomization distribution. Our work addresses this by explicitly decomposing the distribution into these two components. We use Bayesian Optimization to align the center with the target domain, while using entropy maximization to expand the spread in a controlled manner, enabling both robustness and accurate sim-to-real transfer.

3 METHODOLOGY

We train exclusively in a cheaper surrogate simulator $\hat{r}$ and evaluate only in a more expensive target simulator r . Let $\xi \in \mathbb{R}^d$ denote dynamics parameters. Training uses a DR distribution $\nu_\phi(\xi)$ and a policy π_θ . Following DORAEMON Tiboni et al. (2024), we maximize the entropy of ν_ϕ while enforcing a minimum surrogate success probability, where success is defined via a return threshold J_{LB} :

$$G(\theta, \phi) \triangleq \mathbb{E}_{\xi \sim \nu_\phi} \mathbb{E}_{\tau \sim p_{\hat{r}}(\cdot | \pi_\theta, \xi)} [\mathbf{1}\{J_{\hat{r}}(\tau) \geq J_{LB}\}] \geq \alpha \quad (1)$$

The dynamics distribution $\nu_{\mu, \psi}(\xi)$ is parameterized with center $\mu \in \mathbb{R}^{d_c}$ and spread ψ . For fixed μ , DORAEMON adapts (θ, ψ) in $\hat{r}$:

$$\max_{\theta, \psi} \mathcal{H}(\nu_{\mu, \psi}) \quad \text{s.t.} \quad G(\theta, \mu, \psi) \geq \alpha, \quad (2)$$

yielding $\theta^*(\mu)$. BO then optimizes the zero-shot target return:

$$f(\mu) \triangleq \mathbb{E}_{\tau \sim p_r(\cdot | \pi_{\theta^*}(\mu))} [J_r(\tau)], \quad (3)$$

3.1 ALGORITHM

The overall method alternates between entropy-based spread adaptation and BO-based center optimization. This combination allows the method to simultaneously achieve robustness and alignment, improving sim-to-real transfer compared to prior approaches that optimize only one aspect of the distribution.

Algorithm 1 BODREM Algorithm

```

1: procedure BODREM( $\pi, \mu_0, T, b, \hat{r}, r$ )
2:    $\pi_\theta \leftarrow \pi$  with a warm-start at initial  $\mu_0$ 
3:    $\pi^{\text{best}} \leftarrow \pi_\theta$ 
4:    $SR_r^{\text{best}} \leftarrow 0$ 
5:   for iteration = 1, ...,  $T$  do
6:      $\mu_t \leftarrow \text{BO.ASK}$ 
7:      $\nu_\phi \leftarrow$  distribution centered at  $\mu_t$ 
8:     for block = 1, ...,  $b$  do
9:        $\xi \sim \nu_\phi$  ▷ sample dist.  $\xi$  from  $\nu_\phi$ 
10:       $\pi_\theta \leftarrow$  trained model in  $\hat{r}$ 
11:       $\psi_t \leftarrow \arg \max_\psi \mathcal{H}(\nu_{\mu_t, \psi})$ 
12:      subject to  $G(\theta, \mu_t, \psi) \geq \alpha$ 
13:       $(SR_r) \leftarrow \text{EVALUATE-GREEDILY}(\pi_\theta, r)$ 
14:       $\text{BO.TELL}(\mu_t, SR_r)$ 
15:      if  $SR_r > SR_r^{\text{best}}$  then
16:         $SR_r^{\text{best}} \leftarrow SR_r$ 
17:         $\pi^{\text{best}} \leftarrow \pi_\theta$ 
return  $\pi^{\text{best}}, SR_r^{\text{best}}$ 

```

4 EXPERIMENTS

4.1 MOUNTAINCAR TASK

We evaluate five methods (BO, DR, BO+DR, DORAEMON, and BO+DORAEMON; denoted **BORDEM**) on MountainCar under three edge-profile variants (v1, v2, v3).

Fairness framework. We approached this by three steps:

1. **Keep infrastructure fixed.** We used Gymnasium MountainCar-v0 for surrogate training, TinySim for target evaluation, and the same native DQN defaults (replay buffer 10,000, $\epsilon : 1.0 \rightarrow 0.01$ with decay 0.995, batch size 64, target-network sync every 10 episodes).
2. **Equalize compute.** We distributed exactly **3600 training episodes** per method and per variant, changing schedule shape but not total budget.
3. **Average over seeds.** We ran each setting with **10 seeds**.

Environment conditions are: (i) surrogate training with randomized force and gravity, (ii) target evaluation in a fixed TinySim target environment with known target parameters (force=0.0010, gravity=0.0050), and (iii) edge randomization bounds force $\in [0.0007, 0.0018]$, gravity $\in [0.0030, 0.0059]$. The primary metric is `target_solve_rate`, the fraction of target-evaluation episodes that reach the goal within the evaluation horizon.

Baseline (v1). v1 is the reference schedule at fixed 3600 episodes: BO uses (bo_iters=20, bo_train_episodes_per_iter=180), DR uses (dr_total_episodes=3600, dr_eval_every=180), BO+DR uses ($T = 20, K = 180$), DORAEMON uses (adapt_iters=20, blocks=30, episodes_per_block=6), and BORDEM uses ($T = 20$) with (blocks=30, episodes_per_block=6). For BORDEM, we disable extra phases (stabilization_blocks=0, bo_init_random_points=0, center_hold_rounds=1, competence_extra_rounds=0) so the effective training budget remains exactly 3600.

Ablation (v2). v2 keeps the same total budget but changes how training is chunked. For BO/DR/BO+DR this means more frequent outer-loop updates with fewer episodes per update (e.g., BO: 30×120 , BO+DR: $T = 30, K = 120$). For DORAEMON/BORDEM, we keep the outer adaptation count and reduce block count while increasing episodes per block ($20 \times 15 \times 12$). In BORDEM terms, this tests the same number of BO mean-center updates with larger per-block adaptation chunks.

Ablation (v3). v3 shifts toward fewer BO-style mean updates and deeper training per update while preserving 3600 episodes (e.g., BO: 10×360 , BO+DR: $T = 10, K = 360$, DORAE-

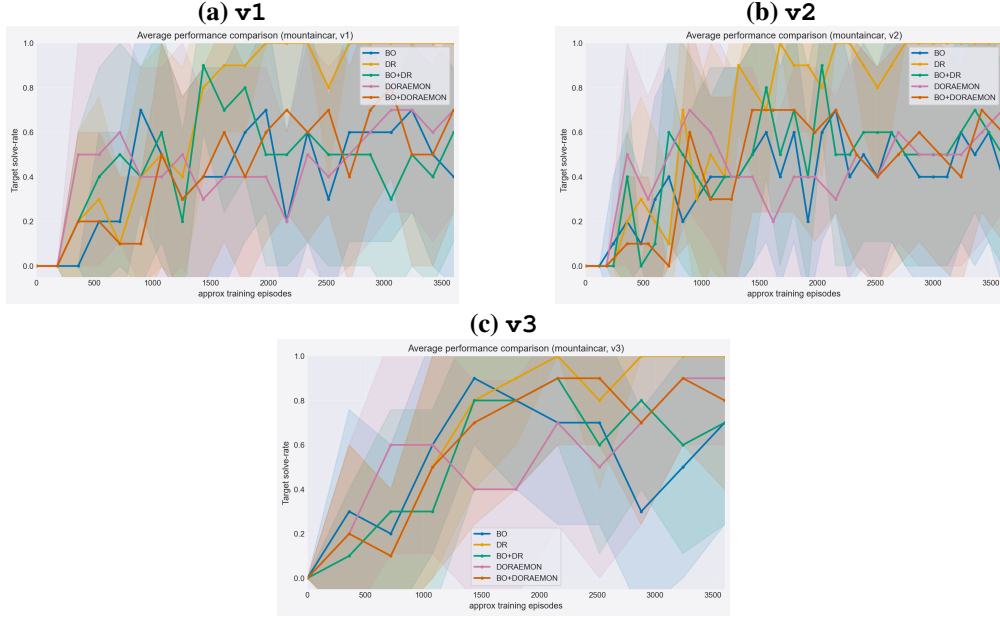


Figure 1: MountainCar target solve-rate versus approximate training episodes (10 seeds), grouped by edge-profile variant.

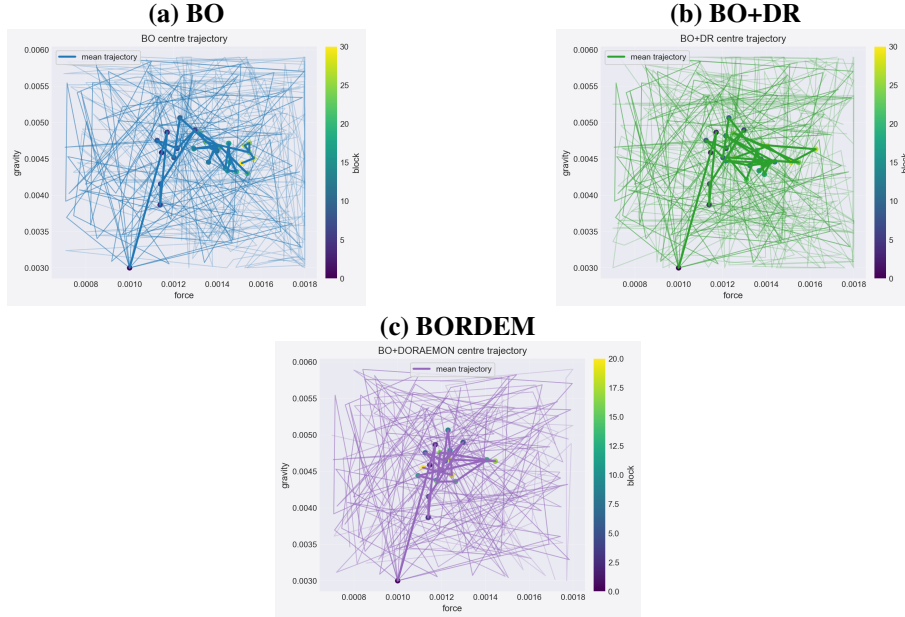


Figure 2: MountainCar center trajectories for BO-based methods (10-seed aggregates). Since the target is known and fixed at ($\text{force}=0.0010$, $\text{gravity}=0.0050$), we can directly inspect center alignment behavior.

MON/BORDEM: $10 \times 15 \times 24$). For BORDEM, this specifically probes the tradeoff between frequent center adaptation and longer robustness adaptation at each selected center.

Figure 1 shows MountainCar learning curves across $v1/v2/v3$. Figure 2 shows BO-center convergence diagnostics for BO, BO+DR, and BORDEM. Table 1 reports *best validation* solve-rate. Detailed episode-allocation and implementation settings are listed in Appendix Tables 6–9. We defer interpretation of comparative performance to Discussion.

Table 1: MountainCar best validation solve-rate (mean $\pm$ std, 10 seeds, 3600 episodes).

Variant	Method	Best Validation
v1	BO	0.70 ± 0.46
v1	DR	1.00 ± 0.00
v1	BO+DR	0.90 ± 0.30
v1	DORAEMON	0.70 ± 0.46
v1	BORDEM	0.80 ± 0.40
v2	BO	0.70 ± 0.46
v2	DR	1.00 ± 0.00
v2	BO+DR	0.90 ± 0.30
v2	DORAEMON	0.70 ± 0.46
v2	BORDEM	0.70 ± 0.46
v3	BO	0.90 ± 0.30
v3	DR	1.00 ± 0.00
v3	BO+DR	0.90 ± 0.30
v3	DORAEMON	0.90 ± 0.30
v3	BORDEM	0.90 ± 0.30

4.2 HALF CHEETAH TASK

For HalfCheetah, we run the same five methods under edge profile ‘v4’ with an episode budget of 1800 and 5 seeds per method.

Fairness framework. We approached this by three steps:

1. **Keep infrastructure fixed.** We used the same method set (BO, DR, BO+DR, DORAEMON, BORDEM), one backend (SB3 SAC), and one evaluation protocol (eval_episodes=5, deterministic eval).
2. **Equalize compute.** We redistributed a fixed **1800-episode** budget across methods (BO: `bo_iters  $\times$  bo_train_episodes_per_iter`, DR: `dr_total_episodes`, BO+DR: `T  $\times$  K`, DORAEMON: `adapt_iters  $\times$  blocks  $\times$  episodes_per_block`, BORDEM: `T  $\times$  blocks  $\times$  episodes_per_block` with extra phases disabled for strict budget control).
3. **Average over seeds.** We ran **5 seeds**.

We selected 1800 episodes for HalfCheetah due to computational cost: in our setup, Gymnasium/MuJoCo training was not GPU-accelerated end-to-end and a single seed took roughly one hour. This constrained the total HalfCheetah run count. In contrast, MountainCar was inexpensive enough to run the full 150-experiment grid (10 seeds $\times$ 5 algorithms $\times$ 3 variants).

Environment conditions are more structured than MountainCar: training randomizes a 7D dynamics vector (six limb masses and ground friction), while target evaluation fixes one hidden target per seed. The randomized parameter bounds are `back_thigh` mass [0.08, 2.99], `back_shin` mass [0.08, 3.08], `back_foot` mass [0.05, 2.08], `front_thigh` mass [0.07, 2.78], `front_shin` mass [0.06, 2.30], `front_foot` mass [0.04, 1.66], and surface friction [0.02, 0.78]. We set `surrogate_success_threshold=0.0` for these runs.

For this task, `mean_episodic_return` is the arithmetic mean of cumulative reward per target-evaluation episode.

Figure 3 shows the overall comparison. Detailed per-method curves are shown together in Appendix Figure 6. Table 2 reports *best validation* return only. Full episode-allocation and implementation settings are listed in Appendix Tables 7–9. We defer interpretation to Discussion.

4.3 MANIPULATION TASK

As a more complex task, we evaluated both baselines and the proposed model on a MuJoCo push-block task, in which a 7-DOF Franka Emika Arm must push a block to a goal position. The physics parameters, block sliding friction and mass, are randomized within bounds during training ($\hat{r}$) but

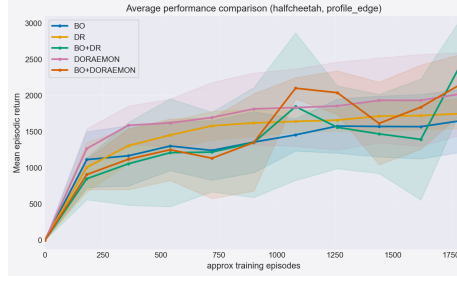


Figure 3: HalfCheetah edge profile comparison (5 seeds, 1800 episodes).

 Table 2: HalfCheetah best validation return (mean $\pm$ std, ‘v4’, 5 seeds, 1800 episodes).

Method	Best Validation
BO	1654.7 $\pm$ 434.8
DR	1752.5 $\pm$ 209.4
BO+DR	2478.2 $\pm$ 619.1
DORAEMON	2025.6 $\pm$ 567.7
BORDEM	2169.9 $\pm$ 405.7

fixed at specific values during evaluation (r). We control task difficulty by adjusting the width of the training bounds; wider bounds entail more robust policies. The reward is a dense signal combining negative distance to the goal, a velocity-toward-goal bonus, and a penalty for the end-effector drifting away from the block. Episodes terminate when the block is within a 5cm threshold of the goal, or after 128 steps.

Fairness Framework. We ensure fairness in three steps:

- **Keep infrastructure fixed.** We used TinySim-MuJoCo environment for all three models. An identical SAC agent architecture (Gaussian Actor with hidden_dim=256, shared replay buffer) was used to ensure fair architectural comparison.
- **Equalize compute.** We distributed exactly **7000 training episodes** per method and per variant, changing schedule shape but not total budget; details are given in Appendix A.3.
- **Average over seeds.** We ran each setting with **5 seeds**.

All three methods were evaluated on easy and hard difficulty settings, with each (method, difficulty) cell averaged over 6 random seeds. On one hand, the easy settings had a surrogate sliding friction $\in [0.6\mu, 1.4\mu]$ with a mass $\in [60g, 200g]$ and corresponding targets of 0.7μ and 0.18 . The hard settings, on the other hand, had a surrogate sliding friction $\in [0.1\mu, 2.5\mu]$ and a mass $\in [20g, 500g]$ and targets of 0.15μ and $0.45gm$ respectively. All methods start from the same SAC warm-start checkpoint to isolate the contribution of each adaptation strategy. All experiments were run on an Nvidia GeForce RTX 5090 GPU.

Baseline (easy). The easy setting constrains physics randomization to a narrow band: friction $\in [0.6, 1.4]$ and mass $\in [0.06, 0.20]$ kg, with a fixed evaluation target of (friction = 0.7, mass = 0.18 kg). The target lies comfortably inside the training distribution, requiring only moderate robustness. BO uses ($T=20$, episodes_per_iter=350) for a budget of 7,000 episodes. DORAEMON uses (blocks=20, episodes_per_block=350), with $\alpha_{\text{target}} = 0.50$, kl_max=0.06, and n_candidates=400. BODREM uses $T = 20$ with (blocks=5, episodes_per_block=70), with $\alpha_{\text{target}} = 0.60$, kl_max=0.05, n_candidates=300, and conc_init_mode=reset, yielding the same 7,000-episode budget. The DR distribution is a product of two independent Beta distributions with initial concentration $\psi_0 = 18$ and bounds $\psi \in [2.2, 220]$ for both parameters.

Ablation (hard). The hard setting widens the randomization bounds to friction $\in [0.1, 2.5]$ and mass $\in [0.02, 0.50]$ kg, placing the evaluation target at (friction= 0.15, mass= 0.45,kg) — near the boundary of the training distribution and far from the warm-start default (friction= 1.0, mass= 0.1,kg). All method configs and episode budgets are held identical to the easy setting, so the only



Figure 4: Manipulation task comparison under the easy setting.

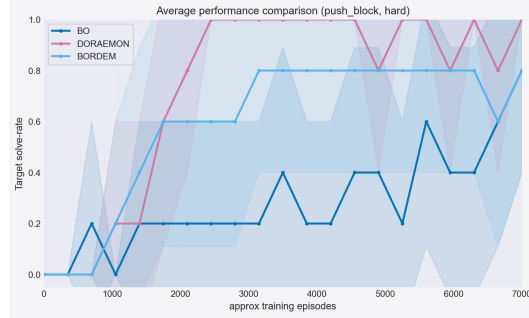


Figure 5: Manipulation task comparison under the hard setting.

difference is the difficulty of the required generalization. This specifically probes each method’s ability to maintain a useful DR distribution and guide the policy toward the target when the physics space is large and the target is an outlier relative to the default dynamics.

Figures 4 and 5 compare the three methods on the manipulation task under the easy and hard settings, respectively. Detailed per-method curves are shown together in Appendix Figures 10 and 11. Table 3 shows the best-validation success rate for push-block.

5 DISCUSSION

We added BO+DR as a control because it is the cleanest ablation of the original idea: keep BO-style center adaptation and add randomization, but without the full BORDEM coupling. In hindsight, that control was important. BO+DR ended up being more competitive than expected, coming in as a close second on MountainCar and the strongest method on HalfCheetah by best validation return.

That result changes how we read the rest of the table. On MountainCar, DR still dominates, with BO+DR next and BORDEM staying competitive with DORAEMON across variants. The center-convergence plot in Figure 2 suggests that this is not because BO-based methods fail to move toward the target; they do. The more likely issue is that, in this simpler setting, broad randomization already captures enough of the target behavior, so the extra structure from center adaptation has limited room to help. The numbers are fairly stable across variants as well: BO+DR reaches 0.90 ± 0.30 in $v1$ and $v2$, while BORDEM stays at 0.80 ± 0.40 in $v1$ and 0.70 ± 0.46 in $v2$, before all of BO/BO+DR/DORAEMON/BORDEM cluster at 0.90 ± 0.30 in $v3$. On HalfCheetah, the picture is different: BO+DR leads at 2478.2 ± 619.1 , BORDEM follows at 2169.9 ± 405.7 , DORAEMON sits at 2025.6 ± 567.7 , and DR/BO trail at 1752.5 ± 209.4 and 1654.7 ± 434.8 . That makes BORDEM look useful, but also suggests that the BO–DORAEMON coupling is not yet tuned as well as it could be.

However, the push-block manipulation task tells a different story. Here, while we did not run BO+DR, among the three remaining methods DORAEMON is the most reliable: all 5 seeds reached the target on both the *easy* and *hard* variants (best-iteration empirical success of 0.42 ± 0.29 Easy,

Table 3: Push Block best empirical success rate at target parameters (mean $\pm$ std, 5 seeds, 21 BO iterations).

Method	Easy	Hard
BO	0.195 ± 0.249	0.399 ± 0.211
DORAEMON	0.423 ± 0.285	0.510 ± 0.160
BORDEM	0.800 ± 0.400	0.800 ± 0.400

0.51 ± 0.16 Hard), whereas BORDEM loses one seed on each difficulty, yielding 0.80 ± 0.40 in both cases, and BO collapses on Easy (0.20 ± 0.25) before partially recovering on Hard (0.40 ± 0.21). The failure pattern is illuminating as BORDEM’s high-variance seed is one in which the BO component locks onto a region before the DR distribution has had time to widen, while BO’s Easy collapse is the expected brittleness of a purely directed search with no robustness cushion. DORAEMON benefits precisely from not having a BO head pulling the distribution off target, so its curriculum is free to concentrate on the minor parameters without being misled.

The clearest takeaway is that alignment and robustness are both real levers, but they do not interact in a uniformly helpful way, and which method wins depends on how well the coupling is tuned for the domain. Pure BO aligns the center, but without enough spread, it is brittle; this shows up on both HalfCheetah and push block. BORDEM adds the spread component back in and is strongest on HalfCheetah, yet on push block, the same coupling costs it one seed per variant, bringing its mean below DORAEMON’s. We read the surprise strength of BO+DR on HalfCheetah and of DORAEMON on push block as two faces of the same observation: whenever the BO–DORAEMON coupling is not well tuned for the task, a simpler method that commits to either directed search or undirected robustness tends to beat the combined one.

At the same time, we should keep the strength of the evidence in perspective. Several of the differences are within one standard deviation, and both HalfCheetah and push block were run with only five seeds per method, with push-block BORDEM’s variance inflated by a single seed where BO–DR coupling misfired. So while the trends are informative, they are not yet strong enough to support definitive superiority claims. They do, however, give a coherent picture: center adaptation is useful, randomization is still necessary, and the remaining gain is probably in better schedule design rather than in adding more complexity for its own sake.

6 CONCLUSION

We presented BORDEM, a center-and-spread domain randomization framework built from BO+DORAEMON, and compared it with BO, DR, DORAEMON, and BO+DR across three MountainCar variants, a compute-limited HalfCheetah setting, and a push-block manipulation task with Easy and Hard variants.

The main empirical message is straightforward. DR remains the strongest choice on MountainCar, BO+DR unexpectedly becomes the strongest on HalfCheetah, DORAEMON is the most reliable on push block (where BORDEM loses one seed per variant to a coupling misfire), and BORDEM clearly improves on pure BO across the board while tracking DORAEMON closely. So the decomposition into center and spread is promising, but it is not automatically better than a simpler control unless the coupling between the two parts is tuned well.

That is why we treat these results as suggestive rather than final. Many of the comparisons sit within one standard deviation, and both HalfCheetah and push block were limited to five seeds, with push-block BORDEM’s variance driven by a single failed seed. The next step is not simply to add more machinery, but to spend more effort on the schedule, coupling, and asymptotic behavior of the BO-and-randomization interaction, and then test it on broader tasks with stronger statistical support.

7 TEAM CONTRIBUTIONS

Suhaima Ahmad - Writing, Proofreading, Related Works
 Fata Asgharznazi - Writing, Github, Experiments: Push Block
 Kevan Busch - Writing, Github, Experiments: MountainCar and HalfCheetah

REFERENCES

- Fabio Muratore, Christian Eilers, Michael Gienger, and Jan Peters. Data-efficient domain randomization with bayesian optimization. *IEEE Robotics and Automation Letters*, 6(2):911–918, April 2021.
- Gabriele Tiboni, Pascal Klink, Jan Peters, Tatiana Tommasi, Carlo D’Eramo, and Georgia Chaltatzaki. Domain randomization via entropy maximization, 2024.
- Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30, 2017. doi: 10.1109/IROS.2017.8202133.
- Zi Wang and Stefanie Jegelka. Max-value entropy search for efficient bayesian optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, volume 70, pp. 3627–3635. PMLR, 2017. URL <https://proceedings.mlr.press/v70/wang17e.html>.

A APPENDIX

A.1 FINAL-CHECKPOINT RESULTS

Main-text tables report *best validation* values. Tables 4 and 5 report final-checkpoint performance to separate endpoint behavior from best-observed validation behavior.

Table 4: MountainCar final checkpoint solve-rate (mean $\pm$ std, 10 seeds, 3600 episodes).

Variant	Method	Final
v1	BO	0.40 $\pm$ 0.49
v1	DR	1.00 $\pm$ 0.00
v1	BO+DR	0.60 $\pm$ 0.49
v1	DORAEMON	0.70 $\pm$ 0.46
v1	BORDEM	0.70 $\pm$ 0.46
v2	BO	0.40 $\pm$ 0.49
v2	DR	1.00 $\pm$ 0.00
v2	BO+DR	0.50 $\pm$ 0.50
v2	DORAEMON	0.70 $\pm$ 0.46
v2	BORDEM	0.60 $\pm$ 0.49
v3	BO	0.70 $\pm$ 0.46
v3	DR	1.00 $\pm$ 0.00
v3	BO+DR	0.70 $\pm$ 0.46
v3	DORAEMON	0.90 $\pm$ 0.30
v3	BORDEM	0.80 $\pm$ 0.40

Table 5: HalfCheetah final checkpoint return (mean $\pm$ std, ‘v4’, 5 seeds, 1800 episodes).

Method	Final
BO	1654.7 $\pm$ 434.8
DR	1752.5 $\pm$ 209.4
BO+DR	2478.2 $\pm$ 619.1
DORAEMON	2025.6 $\pm$ 567.7
BORDEM	2170.0 $\pm$ 405.7

A.2 EXPERIMENTAL SETTINGS

Tables 6–9 summarize the full implementation details used to keep comparisons fair.

Table 6: MountainCar edge schedules by variant (all equivalent to 3600 training episodes).

Method	v1	v2	v3
BO	20×180	30×120	10×360
DR	3600 (eval_every=180)	3600 (120)	3600 (360)
BO+DR	$T = 20, K = 180$	$T = 30, K = 120$	$T = 10, K = 360$
DORAEMON	$20 \times 30 \times 6$	$20 \times 15 \times 12$	$10 \times 15 \times 24$
BORDEM	$20 \times 30 \times 6$	$20 \times 15 \times 12$	$10 \times 15 \times 24$

Table 7: HalfCheetah edge overrides (all equivalent to 1800 training episodes, 5 seeds).

Method	Override
BO	bo_iters=10, bo_train_episodes_per_iter=180
DR	dr_total_episodes=1800, dr_eval_every=180
BO+DR	$T = 10, K = 180$
DORAEMON	adapt_iters=10, blocks=15, episodes_per_block=12
BORDEM	$T = 10, \text{blocks}=15, \text{episodes_per_block}=12, \text{with}$ $\text{stabilization_blocks}=0, \text{bo_init_random_points}=0,$ $\text{center_hold_rounds}=1, \text{competence_extra_rounds}=0$

Table 8: Core implementation defaults used across experiments.

Component	Setting
MountainCar runtime	Native DQN; surrogate training in Gymnasium MountainCar-v0; target evaluation in TinySim.
HalfCheetah runtime	SB3 SAC; HalfCheetah-v4; hidden target sampled once per seed.
DQN defaults	Replay buffer 10,000, batch size 64, $\epsilon : 1.0 \rightarrow 0.01$ with decay 0.995, target-network sync every 10 episodes.
Episode horizon	400 steps per training/evaluation episode.
Evaluation setup	MountainCar metric: target_solve_rate. HalfCheetah metric: mean_episodic_return. Edge runs use deterministic eval with 5 episodes.

Table 9: Method-specific control knobs (edge defaults unless overridden).

Method	Key controls
BO	bo_iters, bo_train_episodes_per_iter, target_eval_episodes; fallback bo_epsilon_reset=0.10.
BO+DR	T, K, B_r, B_r_extra; alpha_low=0.52, alpha_high=0.78; sigma_expand=1.08, sigma_shrink=0.92, sigma_update_interval=4; surrogate_mode=stress.
DORAEMON	adapt_iters, blocks, episodes_per_block, B_r; alpha_target=0.60, kl_max=0.04, n_candidates=300; concentration defaults c_init=18.0, c_min=2.2, c_max=220.0.
BORDEM	T, blocks, episodes_per_block, B_r; freeze_doraemon_mean=True and conc_init_mode=blend by default; optional stabilization/init/hold/competence phases available but disabled in strict-budget sweeps.

A.3 EPISODE-BUDGET BREAKDOWN

- **BO-only**
 - Outer loop: $T = 20$ iterations
 - Training episodes per iteration: 350
 - Total training episodes: $20 \times 350 = 7,000$
 - Target evaluations: 1 (init) + 20 (per iter) = 21 at $B_r = 20$ episodes each = 420 episodes
 - **Total budget: $\sim 7,420$ episodes per seed**
- **DORAEMON-only**
 - Outer loop: blocks = 20
 - Training episodes per block: 350
 - Total training episodes: $20 \times 350 = 7,000$
 - Target evaluations: 1 (init) + 20 (eval every block) = 21 at $B_r = 20$ episodes each = 420 episodes
 - **Total budget: $\sim 7,420$ episodes per seed**
- **BODREM (BO + DORAEMON)**
 - Outer loop: $T = 20$ iterations
 - Inner loop per iteration: blocks = 5
 - Training episodes per block: 70
 - Total training episodes: $20 \times 5 \times 70 = 7,000$
 - Target evaluations: 1 (init) + 20 (per outer iter) = 21 at $B_r = 20$ episodes each = 420 episodes
 - **Total budget: $\sim 7,420$ episodes per seed**
- **Summary**
 - All three methods use **identical total environment interaction**: $\sim 7,420$ episodes per (method, difficulty, seed)
 - Only structural difference: BO explores via outer optimization; DORAEMON via inner DR adaptation; BODREM combines both

A.4 ADDITIONAL HALFCHEETAH DIAGNOSTICS

All five per-method diagnostics are shown on one page for quick visual comparison.

A.5 ADDITIONAL MOUNTAINCAR DIAGNOSTICS

All five per-method summaries are shown for each MountainCar variant using the mean-performance.

A.6 ADDITIONAL MANIPULATION DIAGNOSTICS

The per-method success-rate curves for the easy and hard push-block settings are consolidated on one appendix page each for direct comparison.

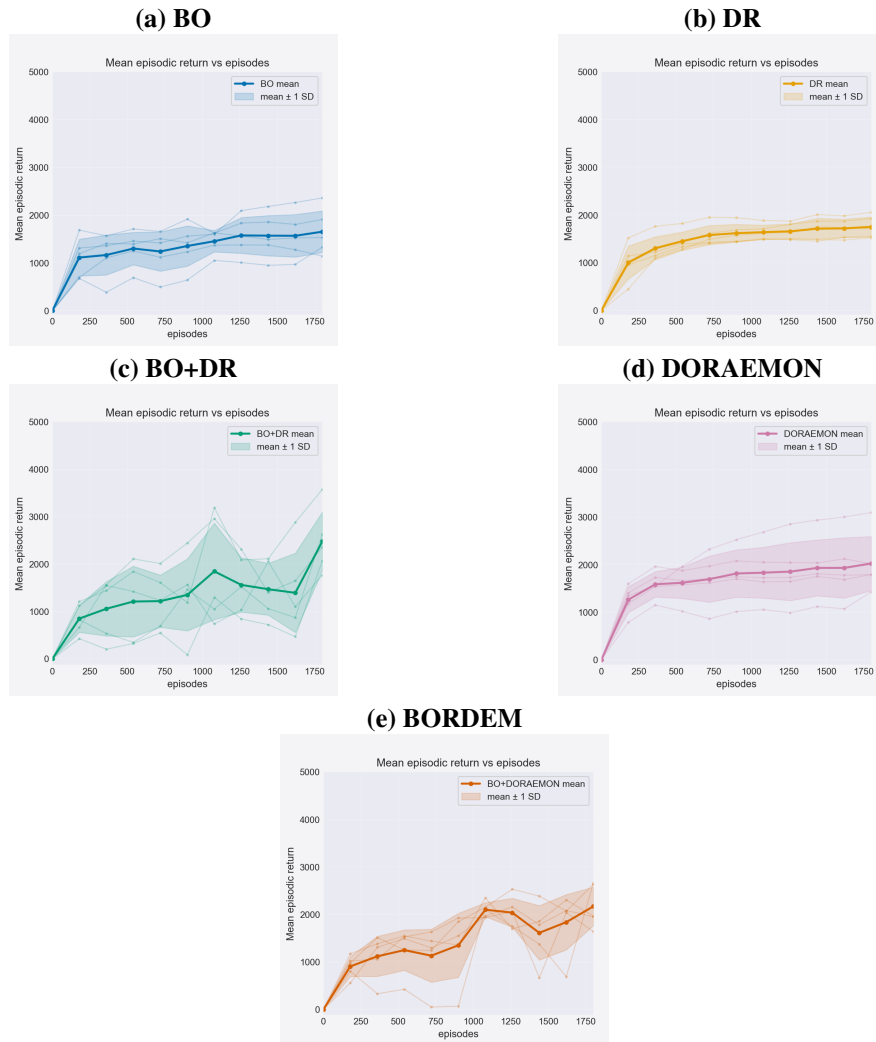


Figure 6: HalfCheetah per-method diagnostics (mean episodic return), consolidated on one appendix page.

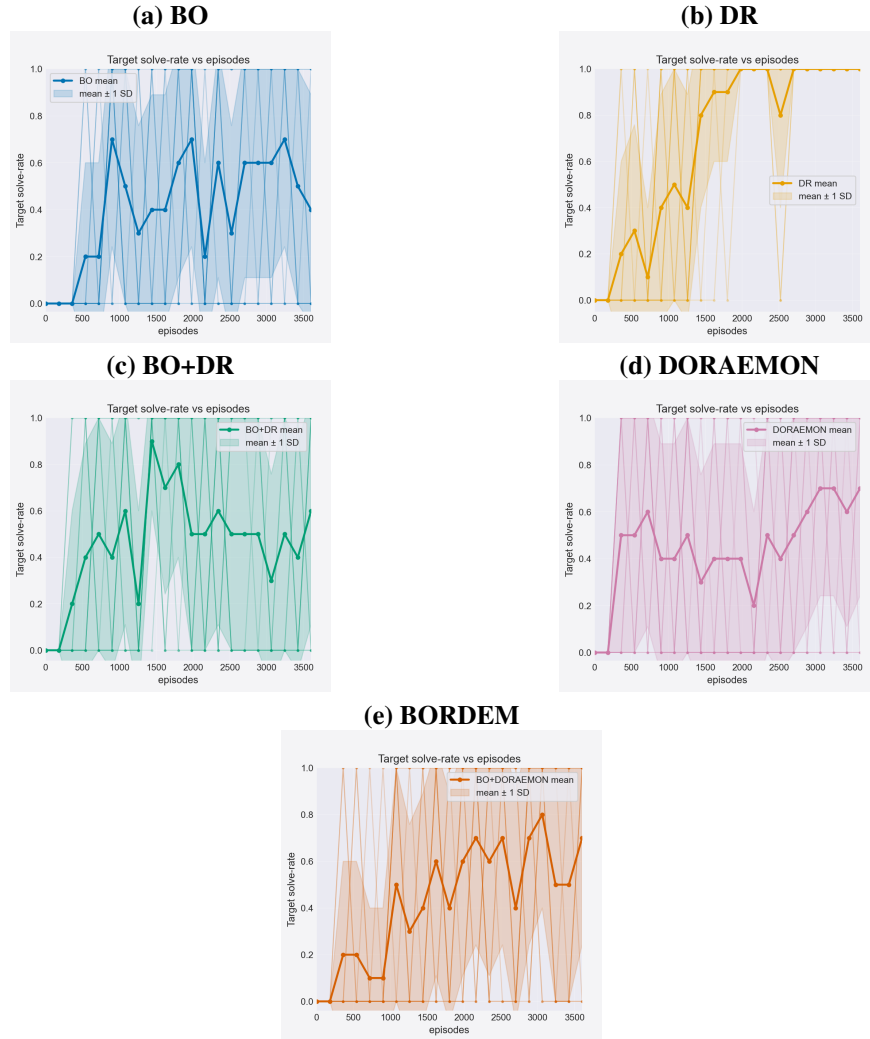


Figure 7: MountainCar v1 per-method summaries .

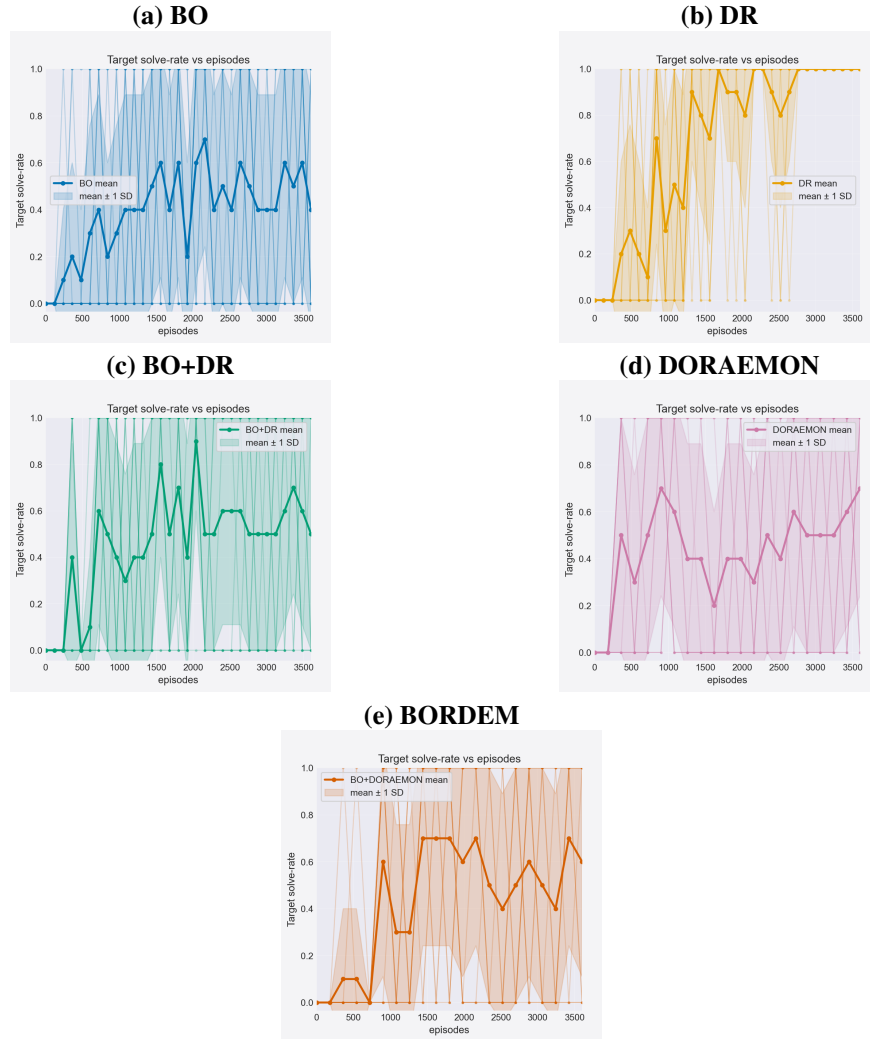


Figure 8: MountainCar v2 per-method summaries .

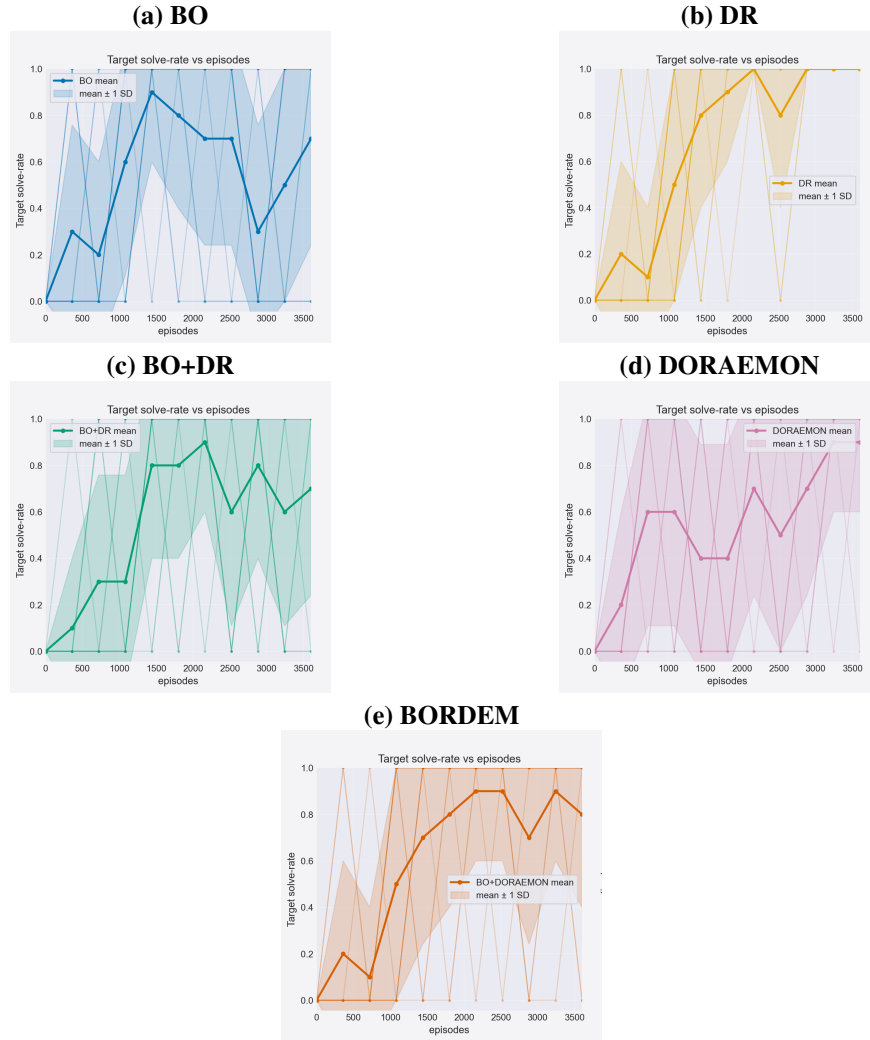


Figure 9: MountainCar v3 per-method summaries .

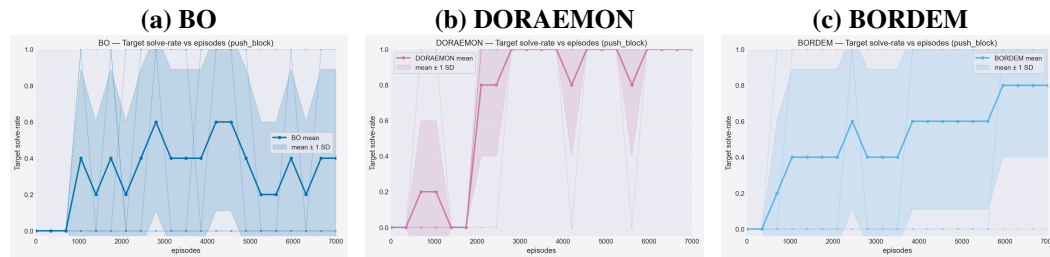


Figure 10: Manipulation per-method diagnostics under the easy setting, consolidated on one appendix page.

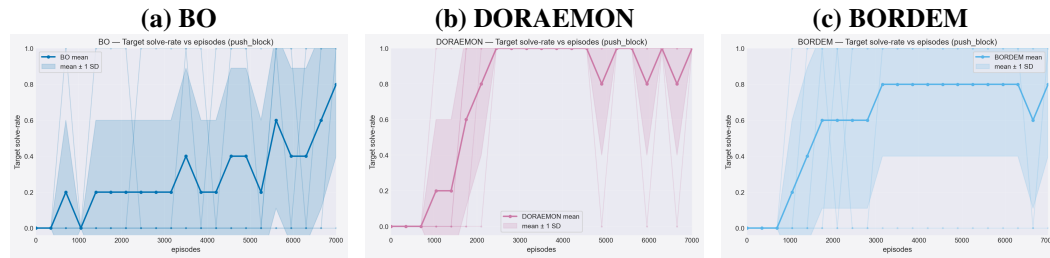


Figure 11: Manipulation per-method diagnostics under the hard setting, consolidated on one appendix page.